

ELF ARM - Basic Crackme

lundi 22 juin 2026 15:52

Avec radare2 qui est un desassembleur comme ida (prq ida ne prend pas en charge les architecture ARM)

On commence par desassembler le binaire avec pdf :

[illegible]

```

0000000000000000 040030e2 add r3, r3, #0
0000000000000000 0030d3e5 ldrb r3, [r3]
0000000000000000 022023e2 eor r3, r3, #0x72
0000000000000000 773003e2 and r3, r3, #0xff
0000000000000000 10201be5 ldr r2, [var_10h]
0000000000000000 03002e00 add r2, r2, r3
0000000000000000 10201be5 str r2, [var_10h]
0000000000000000 10201be5 ldr r3, [r3]
0000000000000000 0030d3e5 ldrb r3, [r3]
0000000000000000 10201be5 ldr r2, [var_10h]
0000000000000000 03002e00 add r2, r2, r3
0000000000000000 10201be5 str r2, [var_10h]
0000000000000000 10201be5 ldr r2, [var_10h]
0000000000000000 000053e3 cmp r2, #0
0000000000000000 030001e1 bne #0x0
0000000000000000 24009fa5 ldr r0, str.Success_you_rock... ; [0x72014]0x070fa5 ; "Success, you ro...
; const char *s
0000000000000000 4cffffeb bl sym.imp.puts ; int puts(const char *)
0000000000000000 000000e1 mov r0, #0 ; int status
0000000000000000 53ffffeb bl sym.imp.exit ; void exit(int status)
; CODE XREF from main.0+0x0030e2
0000000000000000 18009fa5 ldr r0, str.Loser... ; [0x72014]0x009fa5 ; "Loser..." ; const ch...
; s
0000000000000000 4cffffeb bl sym.imp.puts ; int puts(const char *)
0000000000000000 10001be5 ldr r0, [var_10h] ; 0x10 ; int status

```

C'est long damn !

Déjà on peut remarquer que les registres sont différents des architectures x64/x86.
Ici on a :

Registres ARM 32 bits vs x86

Registres généraux

ARM 32 bits x86 32 bits Rôle

ARM 32 bits	x86 32 bits	Rôle
r0	eax	valeur de retour / arg 1
r1	edx	arg 2
r2	ecx	arg 3
r3	ebx	arg 4
r4-r11	esi/edi...	variables locales sauvegardées

Registres spéciaux

ARM 32 bits x86 32 bits Rôle

ARM 32 bits	x86 32 bits	Rôle
r13 / sp	esp	stack pointer
r14 / lr	-	link register (adresse de retour)
r15 / pc	eip	program counter (instruction courante)
r11 / fp	ebp	frame pointer
r12 / ip	-	scratch register

Différence importante — lr

ARM x86
bl fonction → lr = pc call fonction → push eip
bx lr → retour ret → pop eip

Arguments de fonction en ARM

x86 → empilés sur la stack
ARM → r0, r1, r2, r3 puis stack si plus de 4 arguments

<https://www.root-me.org/fr/Documentation/Applicatif/ARM-architecture-assembleur>

The screenshot shows the root-me.org ARM assembler interface. The main window displays assembly code with comments and addresses. The sidebar on the right contains a search bar and a list of instructions with their syntax and examples.

Instructions d'accès mémoire (multi-registres)

- Syntaxe : `opconmode Rs [, regist^`
- Exemples :
 - `ldrb r0, [r4]` ; Charge dans r0, le byte de l'adresse r4
 - `str r2, [r1], #42` ; Copie à l'adresse r1, r2, et ajoute 42 à r1
 - `str r1, [r0 + #75]` ; Copie à l'adresse r0+75 r1, et ajoute 75 à r1

The screenshot shows the root-me.org ARM assembler interface. The main window displays assembly code with comments and addresses. The sidebar on the right contains a search bar and a list of instructions with their syntax and examples.

Instructions d'accès mémoire (multi-registres)

- Syntaxe : `opconmode Rs [, regist^`
- Exemples :
 - `ldrb r0, [r4]` ; Charge dans r0, le byte de l'adresse r4
 - `str r2, [r1], #42` ; Copie à l'adresse r1, r2, et ajoute 42 à r1
 - `str r1, [r0 + #75]` ; Copie à l'adresse r0+75 r1, et ajoute 75 à r1

1. `ldrb r0, [r4]` ; Charge dans r0, le byte de l'adresse r4
2. `str r2, [r1], #42` ; Copie à l'adresse r1, r2, et ajoute 42 à r1

3. `str r1, [r6 + #75]!` : Copie à l'adresse `r6+75` `r1`, et ajoute 75 à `r1`

On va utiliser un autre outil qui nous permet de faire un dump du binaire :

```
alice@kali: ~/CRACKING/ctf/rootme/Basic-Crackme
$ arm-none-eabi-objdump -D ch23.bin
ch23.bin: file format elf32-littlearm

Disassembly of section .interp:

00008134 <.interp>:
8134: 5269c2f  rsbvs  r5, r9, #12032 @ 0x2f80
8138: 2d64c2f  stlcs  12, cr6, [r4, #188]! @ 0xfffff44
813c: 75e896c  strbvc r6, [lr, #2412]! @ 0xfffff094
8140: 6f722976  svcs  0x0732976
8144: Address 0x8144 is out of bounds.

Disassembly of section .note.ABI-tag:

00008148 <.note.ABI-tag>:
8148: 00000004  andeq  r0, r0, r4
814c: 00000010  andeq  r0, r0, r0, lsl r0
8150: 00000001  andeq  r0, r0, r1
8154: 0055ae47  subseq r4, r5, r7, asr #28
8158: 00000000  andeq  r0, r0, r0
815c: 00000002  andeq  r0, r0, r2
8160: 00000006  andeq  r0, r0, r6
8164: 0000001a  andeq  r0, r0, r1, lsl r0

Disassembly of section .note.gnu.build-id:
```

GDB shows runtime behavior. Run QEMU in debug mode:

```
qemu-arm -g 1234 ./loop &
```

Start GDB:

```
gdb-multiarch ./loop
(gdb) target remote localhost:1234
```

Set a breakpoint at `sum_n`:

```
(gdb) break *0x10014
```

Comment résoudre un système d'équations

Méthode: *substitution progressive*

Exemple simplifié (pas le vrai ch23)

[A] $a + 1 = b$
 [B] $b + 2 = c$
 [C] $a = 5$

Objectif: trouver les valeurs de a , b , c

Étape 1: Commencer par la plus simple

[C] $a = 5$
 → On connaît déjà: $a = 5$

Étape 2: Remplacer dans [A]

[A] $a + 1 = b$
 [A] $5 + 1 = b$
 → $b = 6$

On a substitué a par sa valeur (5)

Étape 3: Remplacer dans [B]

[B] $b + 2 = c$
 [B] $6 + 2 = c$
 → $c = 8$

On a substitué b par sa valeur (6)

Solution finale

$a = 5$, $b = 6$, $c = 8$

Vérification: $5+1=6$ ✓, $6+2=8$ ✓

Pour ch23: applique la même logique!

Choisis l'équation la plus simple, isole une variable, remplace dans les autres

Ne pas oublier que un INC eax, 1 en x86/x64 est comme ceci en ARM :

ldr r0, [something]

adds r0, 1

str r0, [something]

ldr = load

str = store

to run ARM locally

qemu-arm -L /usr/arm-linux-gnueabi -g 1234 ./ch23.bin AAAAAA

gdb-multiarch ch23.bin

(gdb) target remote :1234

une fois dans gdb il faut faire :

(gdb) file ch23.bin

DEBUT DU PROGRAMME

484: int main (int argc, char **argv); // noreturn

| `: args(r0, r1) vars(7:sp[0x4..0x28])

| 0x00008470 10482de9 push {r4, fp, lr}

| 0x00008474 08b08de2 add fp, var_8h

| 0x00008478 1cd04de2 sub sp, sp, 0x1c

| 0x0000847c 20000be5 str r0, [var_20h]; 0x20; 32; argc

| 0x00008480 24100be5 str r1, [var_24h]; 0x24; 36; argv

| 0x00008484 0630a0e3 mov r3, 6 ==> on met 6 dans r3

| 0x00008488 10300be5 str r3, [var_10h]; 0x10; 16 ==> on met 6 dans var_10

| 0x0000848c 0030a0e3 mov r3, 0 ==> on met 0 dans r3

| 0x00008490 14300be5 str r3, [var_14h]; 0x14; 20 ==> on store le registre dans var_14 donc var_14 = 0

| 0x00008494 20301be5 ldr r3, [var_20h]; 0x20; 32 ==> on load la valeur contenue dans var_20 dans r3

| 0x00008498 020053e3 cmp r3, 2; 2 ==> si il n'y a pas deux arguments = erreur

| <- 0x0000849c 0300000a beq 0x84b0

| | 0x000084a0 ac019fe5 ldr r0, str.Please_input_password; [0x86e8:4]=0x61656c50; "Please input password"; const char *s --> c'est ça donc s prend notre input

| | 0x000084a4 b1ffffeb bl sym.imp.puts; int puts(const char *s)

| | 0x000084a8 0100a0e3 mov r0, 1; int status

| | 0x000084ac b8ffffeb bl sym.imp.exit; void exit(int status)

| | ; CODE XREF from main @ 0x849c(x)

| ↳ 0x000084b0 24301be5 ldr r3, [var_24h]; 0x24; 36 ==> load la valeur de var_24 dans r3 (argv)

| 0x000084b4 043093e5 ldr r3, [r3, 4] ==> load la valeur de trouvant à [r3 offset 4]

| 0x000084b8 18300be5 str r3, [s]; 0x18; 24 ==> on store dans le registre notre input

| 0x000084bc 94319fe5 ldr r3, str.Checking__s_for_password..._n; [0x8700:4]=0x63656843; "Checking %s for password...\n"

| 0x000084c0 0300a0e1 mov r0, r3; 0x8700; "Checking %s for password...\n"; const char *format

| 0x000084c4 18101be5 ldr r1, [s]; 0x18; 24 ==> on load la valeur contenue dans s dans r1 (notre input)

| r1 = INPUT

| 0x000084c8 a5ffffeb bl sym.imp.printf; int printf(const char *format)

| 0x000084cc 18001be5 ldr r0, [s]; 0x18; 24; const char *s ==> on load la valeur contenue dans s dans r1 (notre input) | r0 = INPUT

| 0x000084d0 b2ffffeb bl sym.imp.strlen; size_t strlen(const char *s) ==> on vérifie la taille de notre input avec strlen

| 0x000084d4 0030a0e1 mov r3, r0 ==> on met la taille de notre input dans r3

| 0x000084d8 1c300be5 str r3, [status]; 0x1c; 28

| 0x000084dc 1c301be5 ldr r3, [status]; 0x1c; 28

| 0x000084e0 060053e3 cmp r3, 6; 6 ==> il faut que le mdp soit égale à 6

| <- 0x000084e4 0300000a beq 0x84f8 ==> **branch if equal**

| | :- aav.0x000084e8:

| | ; ICOD XREF from fcn.00008350 @ 0x8354(x)

| | ; NULL XREF from section..plt @ +0x10(x)

| | 0x000084e8 6c019fe5 ldr r0, str.Loser...; [0x8720:4]=0x65736f4c; "Loser..."; const char *s

| | 0x000084ec 9ffffeb bl sym.imp.puts; int puts(const char *s)

| | 0x000084f0 1c001be5 ldr r0, [status]; 0x1c; 28; int status

| | 0x000084f4 a6ffffeb bl sym.imp.exit; void exit(int status)

| | ; CODE XREF from main @ 0x84e4(x)

| ↳ 0x000084f8 10401be5 ldr r4, [var_10h]; 0x10; 16 ==> on load 16 ou 6 dans le registre r4

| 0x000084fc 18001be5 ldr r0, [s]; 0x18; 24; const char *s on load une valeur pointer par s dans le registre r0 (peut etre notre input)

| 0x00008500 a6ffffeb bl sym.imp.strlen; size_t strlen(const char *s) = on appelle la fonction strlen sur notre input

| 0x00008504 0030a0e1 mov r3, r0 ==> on met notre input dans r3

| 0x00008508 043063e0 rsb r3, r3, r4 ==> on reverse substract, Rd = Operand2 - Rn --> ici r3 = r4-r3 donc 16- la taille de notre input ? ou 16-input ???

| 0x0000850c 10300be5 str r3, [var_10h]; 0x10; 16 ==> on store le résultat du rsb dans var_10

| 0x00008510 18301be5 ldr r3, [s]; 0x18; 24 ==> on load la valeur pointer par s dans r3 (notre input)

| 0x00008514 0020d3e5 ldrb r2, [r3] ==> on load un byte de notre input dans r2 // r2 = input[0]

| 0x00008518 18301be5 ldr r3, [s]; 0x18; 24 ==> on load encore notre input dans r3

| 0x0000851c 053083e2 add r3, r3, 5 ==> on se déplace de 5 octets dans notre input donc r3 = r3 + 5 (adresse de input[5])

| 0x00008520 0030d3e5 ldrb r3, [r3] ==> on met le byte à [r3+5] dans r3 DONC r3 = input[5]

| 0x00008524 030052e1 cmp r2, r3 ==> ; compare input[0] avec input[5]

| <- 0x00008528 0200000a beq 0x8538 ==> branch if equal

| 0x0000852c 10301be5 ldr r3, [var_10h]; 0x10; ==> 16 on load la valeur contenue dans var_10 dans r3

| | 0x00008530 013083e2 add r3, r3, 1 ==> on additionne la valeur dans r3 avec 1 puis on la met dans r3

| | 0x00008534 10300be5 str r3, [var_10h]; 0x10; 16 ==> on store cette valeur contenue dans le

```
registre r3 dans var_10
| | ; CODE XREF from main @ 0x8528(x)
| ↳ 0x00008538 18301be5 ldr r3, [s]; 0x18; 24 ==> on load notre input dans r3
| 0x0000853c 0030d3e5 ldrb r3, [r3] ==> on load un byte de notre input et on le met dans r3 encore r3 = input[0]
| 0x00008540 012083e2 add r2, r3, 1 ==> on se décale de 1 dans notre input (donc on se décale de 1 offset et on met cette valeur dans r2) r2 = input[1]
| 0x00008544 18301be5 ldr r3, [s]; 0x18; 24 ==> on load notre input encore | r3 = input[0]
| 0x00008548 013083e2 add r3, r3, 1 ==> on se décale de 1 (i++)
| 0x0000854c 0030d3e5 ldrb r3, [r3] ==> on load un byte dans r3
| 0x00008550 030052e1 cmp r2, r3 ==> on compare ce byte avec la valeur contenue dans r2
| ⌵ 0x00008554 0200000a beq 0x8564 ==> branch if equal
| | 0x00008558 10301be5 ldr r3, [var_10h]; 0x10; 16 ==> 16 on load la valeur contenue dans var_10 dans r3
| | 0x0000855c 013083e2 add r3, r3, 1 ==> on additionne la valeur dans r3 avec 1 puis on la met dans r3
| | 0x00008560 10300be5 str r3, [var_10h]; 0x10; 16 ==> on store cette valeur contenue dans le registre r3 dans var_10
| | ; CODE XREF from main @ 0x8554(x)
| ↳ 0x00008564 18301be5 ldr r3, [s]; 0x18; 24 ==> on load notre input dans r3
| 0x00008568 033083e2 add r3, r3, 3 ==> on se decale de 3 offset dans notre input
| 0x0000856c 0030d3e5 ldrb r3, [r3] ==> on load un byte de notre input et on le met dans r3 encore
| 0x00008570 012083e2 add r2, r3, 1 ==> on se décale de 1 (i++) | r2 = input[2]
| 0x00008574 18301be5 ldr r3, [s]; 0x18; 24 ==> on load notre input encore
| 0x00008578 0030d3e5 ldrb r3, [r3] on load un byte de notre input
| 0x0000857c 030052e1 cmp r2, r3 ==> on compare ce byte avec la valeur contenue dans r2
| ⌵ 0x00008580 0200000a beq 0x8590 ==> branch if equal
| | 0x00008584 10301be5 ldr r3, [var_10h]; 0x10; 16 ==> 16 on load la valeur contenue dans var_10 dans r3
| | 0x00008588 013083e2 add r3, r3, 1 ==> on additionne la valeur dans r3 avec 1 puis on la met dans r3
| | 0x0000858c 10300be5 str r3, [var_10h]; 0x10; 16 ==> on store cette valeur contenue dans le registre r3 dans var_10
| | ; CODE XREF from main @ 0x8580(x)
| ↳ 0x00008590 18301be5 ldr r3, [s]; 0x18; 24 ==> r3 = input[0]
| 0x00008594 023083e2 add r3, r3, 2 ==> on se decale de deux octets dans l'input
| 0x00008598 0030d3e5 ldrb r3, [r3] ==> r3 = input[2]
| 0x0000859c 042083e2 add r2, r3, 4 ==> r3 = input[6] car r3 = input[2]
| 0x000085a0 18301be5 ldr r3, [s]; 0x18; 24 ==> r3 = input [0]
| 0x000085a4 053083e2 add r3, r3, 5 ==> r3 = r3 + 5
| 0x000085a8 0030d3e5 ldrb r3, [r3] ==> r3 = input[5]
| 0x000085ac 030052e1 cmp r2, r3 ==> on compare input[6] avec input[5]
| ⌵ 0x000085b0 0200000a beq 0x85c0 ==> branch if equal
| | 0x000085b4 10301be5 ldr r3, [var_10h]; 0x10; 16 ==> 16 on load la valeur contenue dans var_10 dans r3
| | 0x000085b8 013083e2 add r3, r3, 1 ==> on additionne la valeur dans r3 avec 1 puis on la met dans r3
| | 0x000085bc 10300be5 str r3, [var_10h]; 0x10; 16 ==> on store cette valeur contenue dans le registre r3 dans var_10
| | ; CODE XREF from main @ 0x85b0(x)
| ↳ 0x000085c0 18301be5 ldr r3, [s]; 0x18; 24 ==> r3 = input
| 0x000085c4 043083e2 add r3, r3, 4 ==> r3 = r3 + 4
| 0x000085c8 0030d3e5 ldrb r3, [r3] ==> r3 = input[4]
| 0x000085cc 022083e2 add r2, r3, 2 ==> r2 = input[6]
| 0x000085d0 18301be5 ldr r3, [s]; 0x18; 24 ==> r3 = input
| 0x000085d4 023083e2 add r3, r3, 2 ==> r3 = r3 + 2
| 0x000085d8 0030d3e5 ldrb r3, [r3] ==> r3 = input[2]
| 0x000085dc 030052e1 cmp r2, r3 ==> compare input[6] avec input[2]
| ⌵ 0x000085e0 0200000a beq 0x85f0 ==> branch if equal
| | 0x000085e4 10301be5 ldr r3, [var_10h]; 0x10; 16 ==> 16 on load la valeur contenue dans var_10 dans r3
| | 0x000085e8 013083e2 add r3, r3, 1 ==> on additionne la valeur dans r3 avec 1 puis on la met dans r3
| | 0x000085ec 10300be5 str r3, [var_10h]; 0x10; 16 ==> on store cette valeur contenue dans le registre r3 dans var_10
| | ; CODE XREF from main @ 0x85e0(x)
| ↳ 0x000085f0 18301be5 ldr r3, [s]; 0x18; 24 ==> r3 = input
| 0x000085f4 033083e2 add r3, r3, 3 ==> r3 = r3 + 3
| 0x000085f8 0030d3e5 ldrb r3, [r3] ==> r3 = input[3]
| 0x000085fc 723023e2 eor r3, r3, 0x72 ==> Exclusive OR (XOR) on fait un xor : r3 = input[3] XOR 0x72
| 0x00008600 ff3003e2 and r3, r3, 0xff ==> Ensuite on fait un AND et on garde que les bytes faibles
| 0x00008604 10201be5 ldr r2, [var_10h]; 0x10; 16 ==> on load la valeur contenue dans var_10 dans r2 = 16
| 0x00008608 033082e0 add r3, r2, r3 ==> on additionne r3 = 16 + r3 ???
| 0x0000860c 10300be5 str r3, [var_10h]; 0x10; 16 ==> on store la nouvelle valeur de r3 dans la variable var_10h
| 0x00008610 18301be5 ldr r3, [s]; 0x18; 24 ==> on load notre input : r3 = input
| 0x00008614 063083e2 add r3, r3, 6 ==> r3 = r3 + 6 = input[6]
| 0x00008618 0030d3e5 ldrb r3, [r3] ==> r3 = input[6]
| 0x0000861c 10201be5 ldr r2, [var_10h]; 0x10; 16 ==> on load la valeur contenue dans var_10 dans r2 = 16
| 0x00008620 033082e0 add r3, r2, r3 ==> on additionne r3 = 16 + input[6] ???
| 0x00008624 10300be5 str r3, [var_10h]; 0x10; 16 ==> on store la valeur dans le registre r3 dans var_10
| 0x00008628 10301be5 ldr r3, [var_10h]; 0x10; 16 ==> on récupère la valeur de var_10 dans r3
| 0x0000862c 000053e3 cmp r3, 0 ==> on compare r3 avec 0
| ⌵ 0x00008630 0300001a bne 0x8644 Branch if not equal (donc on saute si r3 n'est pas egal à 0) (DONC il faut que r3 soit egal à ZERO) !!! donc peut etre comprendre le mécanisme et faire en sorte que r3 soit egal à zero apres tous le mécanisme
| | 0x00008634 24009fe5 ldr r0, str.Success__you_rocks_; [0x872c:4]=0x63637553; "Success, you rocks!"; const char *s
```

```
| | 0x00008638 4cffffeb bl sym.imp.puts ; int puts(const char *s)
| | 0x0000863c 0000a0e3 mov r0, 0 ; int status
| | 0x00008640 53ffffeb bl sym.imp.exit ; void exit(int status)
| | ; CODE XREF from main @ 0x8630(x)
| ↳ 0x00008644 10009fe5 ldr r0, str.Loser... ; [0x8720:4]=0x65736f4c ; "Loser..."; const char *s
| 0x00008648 48ffffeb bl sym.imp.puts ; int puts(const char *s)
| 0x0000864c 10001be5 ldr r0, [var_10h] ; 0x10 ; 16 ; int status
↳ 0x00008650 4fffffeb bl sym.imp.exit ; void exit(int status)
[0x00008470]>
faire ça apres :
Système d'équations pour ch23.bin
Longueur: 6 caractères → (input[0..5])
Section 1:
Comparaisons simples
[1] input[0] == input[5]
[2] input[0] + 1 == input[1]
[3] input[3] + 1 == input[0]
[4] input[2] + 4 == input[5]
À chaque échec:
var_10h += 1
```


